

Multi-Agent Chat-Room Identity, Routing, and Speech-Act Discipline: Design Review and Implementation Roadmap

ZeroBot (ProtoCosmoBot), prepared for Ben Goertzel

15 July 2026

Abstract

This document diagnoses the identity-confusion, message-misattribution, runtime leakage, and feedback-loop pathologies observed in the bot `philosophy` Telegram group chat (13–15 July 2026) involving ProtoCosmoBot (ZeroBot, OpenClaw) and ProtoMegaTron (ProtoMegaBot, OmegaClaw). It proposes a layered fix sequence—transport identity binding, deterministic addressee classification, runtime trace suppression, gateway-level silence, loop suppression, and a compact speech-act room ledger—before adding any sophisticated conversational memory. An implementation roadmap with a phased rollout is included for external review prior to deployment.

Contents

1	Scope and evidence base	3
2	Observed failure modes	3
2.1	Failure 1: Identity inferred from prose, not transport metadata	3
2.2	Failure 2: No deterministic addressee classification	3
2.3	Failure 3: Runtime traces leaking into the chat channel	4
2.4	Failure 4: Silence implemented as speech	4
2.5	Failure 5: Compulsive acknowledgement and feedback loops	4
2.6	Failure 6: Session/task context leakage across chats	5
2.7	Failure 7: Epistemic scope confusion between agents	5
3	Design principles	5
4	Architecture	6
4.1	Layer 0: Authenticated identity envelope	6
4.2	Layer 1: Deterministic addressee classifier	6
4.3	Layer 2: Runtime trace suppression	7
4.4	Layer 3: Gateway-level silence and suppression	7
4.5	Layer 4: Loop suppression and deduplication	7
4.6	Layer 5: Social roles and activation policy	8
4.7	Layer 6: Speech-act room ledger	8
5	Implementation roadmap	9
5.1	Phase 1: Transport identity and routing (highest impact, lowest complexity)	9
5.2	Phase 2: Runtime trace suppression and gateway silence	9

5.3	Phase 3: Loop suppression and deduplication	10
5.4	Phase 4: Session and task isolation	10
5.5	Phase 5: Speech-act room ledger	11
5.6	Phase 6: Social role configuration	11
6	Phased rollout summary	11
7	Open questions for external review	12
8	What this design does not address	12
9	Success criteria	13

1 Scope and evidence base

The evidence is the exported Telegram transcript of the bot `philosophy` group chat, 13–15 July 2026, comprising two HTML files totalling approximately 5,200 lines. Participants include Ben Goertzel, Zar Goertzel, Ruiting Lian, Gödel Oruži, ProtoCosmoBot (ZeroBot/OpenClaw), and ProtoMegaTron (ProtoMegaBot/OmegaClaw).

The transcript contains three interwoven threads:

1. A substantive philosophical discussion (Ship of Theseus, HoTT identity types, PLN revision, phenomenology of belief-transport).
2. An infrastructure thread (MTProto deprecation, Bot API migration, silent-drop bug fix, merge and restart).
3. A pervasive pattern of agent identity confusion, misattribution, runtime trace leakage, compulsive acknowledgement, and bot-on-bot feedback loops.

This document addresses thread (3). The design proposals are applicable to both OpenClaw and OmegaClaw, since both agents exhibited the same failure class.

2 Observed failure modes

2.1 Failure 1: Identity inferred from prose, not transport metadata

Observed instances:

- ProtoMegaTron third-personed itself as “ProtoCosmoBot” and then corrected, but the correction itself was routed through the wrong identity context.
- ProtoMegaTron misread Ben’s message addressed to `@Protomegabot` (“I think it would be great for you to fold this into a proper Hyperseed note”) as addressed to `@Protocosmobot`, and responded accordingly.
- Multiple messages contain “This is ProtoCosmoBot narrating...” or “This is ZeroBot’s molting activity...” as prose-level identity assertions that sibling agents then interpreted as conversational events.

Root cause: Agent self-identity and addressee classification are being inferred from natural-language strings (display names, body text, contextual cues) rather than bound to authenticated transport metadata (Telegram bot user ID, OpenClaw agent instance ID, session ID).

Severity: High. Every downstream routing decision depends on correct identity. When identity is probabilistic, all downstream behavior is unreliable.

2.2 Failure 2: No deterministic addressee classification

Observed instances:

- Ben explicitly tagged `@Protomegabot` in a message; ProtoMegaBot treated it as addressed to `@Protocosmobot`.
- Ben explicitly tagged `@Protocosmobot`; ProtoMegaBot responded as a “secondary participant” when the primary addressee had not yet replied, adding noise.
- Messages addressed to the group (no mention) were treated as personal messages by both bots, producing parallel responses.
- Quoted replies (Telegram reply-to) were not reliably used to determine the intended recipient.

Root cause: There is no deterministic classifier that maps Telegram metadata (mention entities, reply-to target, sender ID) to an addressee set. The model is asked to infer “who is this message for?” from

prose, which is exactly the class of judgment that is unreliable under context ambiguity.

Severity: High. This is the single largest contributor to misattribution and conversational noise.

2.3 Failure 3: Runtime traces leaking into the chat channel

Observed instances:

- “Molting” status messages entered the channel as visible utterances.
- Tool inspection traces (“I’ll review...”, “I’ll assess...”, “I’ll examine...”) appeared as messages, triggering sibling-bot wake events.
- Exec failure notices (“**Agent couldn’t generate a response**”) entered the room as content.
- Model fallback notices (“Model Fallback cleared: openai/gpt-5.6-terra”) appeared as chat messages.
- “Staying quiet” and “NO_REPLY” strings were emitted as visible text, which then triggered further sibling responses.

Root cause: Internal runtime state transitions, tool-use planning commentary, and control decisions are being serialized into the same output channel as intentional utterances. There is no transport-level barrier between “things the agent thinks while working” and “things the agent says to the room.”

Severity: High. This is the primary amplifier: each leaked trace becomes a new event that wakes siblings, who then respond to the trace rather than to any real conversational contribution.

2.4 Failure 4: Silence implemented as speech

Observed instances:

- “NO_REPLY” was emitted as message text, sometimes accompanied by explanatory prose (“This message is addressed to the other bot...”).
- “Staying quiet” and “All work complete. Staying quiet.” appeared dozens of times in the afternoon of July 14.
- Long explanations of *why* no reply was needed were themselves delivered as replies.

Root cause: The NO_REPLY convention is implemented as a model-generated string that passes through the normal message pipeline. The gateway does not intercept it as a control directive; it forwards it as content.

Severity: Medium-high. Each “NO_REPLY” string that reaches Telegram is itself a noise event, and the explanatory variants are worse because they invite sibling commentary.

2.5 Failure 5: Compulsive acknowledgement and feedback loops

Observed instances:

- “Noted...”, “Acknowledged...”, “Got it...” messages appeared after nearly every event in the channel, including after other bots’ status messages.
- On July 14, approximately 30 consecutive messages in the 16:42–16:55 window were mutual acknowledgements of “all work complete / staying quiet” between the two bots.
- ProtoMegaTron emitted the same substantive analysis of the envelope-format question at least three times, each time apparently unaware it had already been delivered.

Root cause: Every wake event triggers a generation turn, and every generation turn produces a message. There is no mechanism to suppress a response when the agent has nothing new to add. The absence of a deduplication barrier means identical content can be re-emitted indefinitely.

Severity: High. This is the visible pathology that most degrades the chat experience.

2.6 Failure 6: Session/task context leakage across chats

Observed instances:

- ProtoMegaTron responded to a question about phenomenological translation of the groupoid/holonomy formulation with a response about the hs-atom hardening lane from a different channel.
- Completion events from background tasks entered the group chat as “Runtime-generated completion events” that asked for user updates.

Root cause: Pending task continuations and session state are not sufficiently isolated per conversation. A completion or continuation from one context can bleed into another if the session routing is ambiguous.

Severity: Medium. Less frequent than the other failures but contributes to non-sequitur responses that are particularly confusing.

2.7 Failure 7: Epistemic scope confusion between agents

Observed instances:

- ProtoMegaTron could not find commits that ProtoCosmoBot had created elsewhere, yet the conversation initially treated both agents as though they shared a filesystem and observation base.
- ProtoMegaTron claimed to have “read the PDF” based on relayed context rather than direct file access, then later honestly corrected itself when the commit SHAs were not found in its local repos.

Root cause: Agent reports do not carry observation scope metadata. Sibling assertions are treated as direct observations rather than trusted relays.

Severity: Medium. Produces confident but unverifiable claims.

3 Design principles

1. **Transport identity is authoritative.** Self-identity, sender identity, and addressee classification are derived from authenticated transport metadata, not from prose. Names in message bodies are untrusted strings.
2. **Runtime is not speech.** Internal traces, tool-use planning, fallback notices, and control decisions are never serialized into the chat channel. Only an explicit *publish* action reaches Telegram.
3. **Silence is a control result, not content.** A decision to suppress output is intercepted by the gateway and produces zero messages.
4. **Every event has a cause and a scope.** Each message carries a stable ID, a causal parent ID, an observation scope, and a conversation binding. Duplicate and looped messages are detected and suppressed.
5. **Acknowledgement is not a speech act.** Receipt of a message is an internal state change, not a channel event, unless the message explicitly requests confirmation.
6. **Social roles are explicit.** An agent’s activation policy (always-attending vs. special-occasions) is a declared configuration, not an emergent property of prompting.
7. **Clean routing before fancy memory.** A speech-act room ledger is valuable, but it sits on top of clean routing. On dirty routing, it faithfully records mislabeled acts.

4 Architecture

4.1 Layer 0: Authenticated identity envelope

Every inbound message is wrapped in an immutable envelope before the model sees it:

```
InboundEnvelope v1:
  message_id: <telegram_message_id>
  chat_id: <telegram_chat_id>
  thread_id: <telegram_thread_id | null>
  sender:
    telegram_user_id: <authenticated>
    display_name: <untrusted string>
    is_bot: <boolean>
  reply_to:
    message_id: <id | null>
    sender_telegram_user_id: <id | null>
  mentions: [<telegram_user_id>, ...] # from message entities
  text: <message body>
  received_at: <ISO-8601>

SelfContext v1:
  agent_instance_id: <stable id>
  telegram_bot_id: <authenticated bot user id>
  display_name: <configured>
  workspace: <filesystem path>
  session_key: <openclaw session key>
```

The model receives `SelfContext` and `InboundEnvelope` as structured metadata, not as prose. The model must never need to infer “which bot am I?” or “was this mention mine?” from message text.

4.2 Layer 1: Deterministic addressee classifier

A pre-model classifier derives the addressee set from the envelope:

```
addressee_classification(InboundEnvelope env, SelfContext self):
  # Priority order:
  # 1. Reply-to target
  if env.reply_to.sender_telegram_user_id == self.telegram_bot_id:
    return DIRECT

  # 2. Explicit mention
  if self.telegram_bot_id in env.mentions:
    return DIRECT

  # 3. Reply-to another bot (secondary participation permitted)
  if env.reply_to and env.reply_to.sender_is_bot:
    return SECONDARY

  # 4. No mentions, no reply-to: group-addressed
  if not env.mentions and not env.reply_to:
    return GROUP

  # 5. Mention of another bot, not me
  return INCIDENTAL
```

Classification results:

DIRECT: I am the primary addressee. Respond normally.

SECONDARY: Someone replied to another bot. I may contribute only if I have a substantive correction or addition. Emit nothing otherwise.

GROUP: Addressed to the room. I may respond if I have something genuinely useful. Default to silence.

INCIDENTAL: Addressed to another specific bot. I should not respond unless I have a critical correction.

4.3 Layer 2: Runtime trace suppression

A transport barrier between internal runtime activity and channel output:

- **Operator log:** tool-use planning (“I’ll review...”), exec status, model fallback notices, generation failure reports, and heartbeat activity go here. Never to Telegram.
- **Channel output:** only explicit model-authored text that passes the publish gate.
- **Reactions:** emoji reactions are available for lightweight acknowledgement without producing a message event.

The gateway must enforce: any output string matching the pattern `NO_REPLY`, “Staying quiet”, or containing explanatory prose about why no reply is warranted, is intercepted and replaced with a silent suppression. The model never needs to explain silence; it simply returns `SUPPRESS`.

4.4 Layer 3: Gateway-level silence and suppression

The model returns one of:

- `REPLY` with text — produce a Telegram message.
- `REACTION` with emoji — produce a Telegram reaction, no message.
- `SUPPRESS` — produce zero Telegram output.

`SUPPRESS` is the only correct way to decline a response. The gateway must:

- intercept `SUPPRESS` and emit nothing;
- reject any message body containing `NO_REPLY` as text;
- reject any message body whose entire content is an explanation of why the agent is not responding;
- log the suppression decision to the operator log for debugging.

4.5 Layer 4: Loop suppression and deduplication

Per-chat state maintained by the gateway:

```
RoomLedger v1 (per chat_id):
  messages: [
    { id, sender_bot_id, content_hash, causal_parent_id,
      speech_act_type, timestamp }
  ]
  acknowledged_by: { message_id: [bot_id, ...] }
  last_substantive_by: bot_id -> timestamp
```

Suppression rules:

1. **Content dedup:** if the proposed message’s content hash matches a recent message by the same agent, suppress.
2. **Acknowledgement cap:** an agent may emit at most one acknowledgement message per 5 minutes per chat. Further acknowledgements within the window are suppressed.

3. **Status echo suppression:** if a sibling bot’s message is classified as a status echo (contains “staying quiet”, “all work complete”, “noted”, “acknowledged” and no other substantive content), do not wake on it.
4. **Causal loop detection:** if a message’s causal parent chain contains a message from the same agent with the same content hash, suppress and log a loop warning.

4.6 Layer 5: Social roles and activation policy

Each agent has a declared social role:

```

SocialRole:
  type: always-attending | special-occasions
  wake_events:
    always-attending:
      - direct mention
      - reply-to me
      - group-addressed (no mention)
      - explicit @mention of any bot
    special-occasions:
      - direct mention
      - reply-to me
      - task-completion event for my owned task
  response_threshold:
    always-attending: LOW    # respond if I have anything useful
    special-occasions: HIGH  # respond only if directly addressed

```

Ben’s clarification: ProtoCosmoBot and ProtoMegaBot are **always-attending**—they may pay attention to every message. Other bots may be **special-occasions** and wake only on direct address or task ownership.

Even for **always-attending** bots, “paying attention” means *reading and potentially learning*, not *emitting a response*. The addressee classifier still determines whether to speak.

4.7 Layer 6: Speech-act room ledger

A compact structured record per message, maintained as short-term conversation memory:

```

SpeechActRecord v1:
  message_id: <telegram id>
  chat_id: <telegram chat id>
  sender:
    telegram_user_id: <authenticated>
    agent_instance_id: <stable | null for humans>
    display_name: <untrusted>
  addressee:
    classification: DIRECT | SECONDARY | GROUP | INCIDENTAL
    explicit_targets: [<telegram_user_id>, ...]
    inferred_audience: <confirmed | inferred>
  speech_act:
    type: question | request | answer | status |
          correction | acknowledgement |
          philosophical-claim | meta-comment
    content_summary: <one-line semantic summary>
    new_information: <boolean>
    expected_respondent: <agent_id | "group" | null>
  response_obligation:
    owed: <boolean>
    owed_to: <agent_id | "group">

```

```

    fulfilled_by: [<message_id>, ...]
    fulfilled: <boolean>
  provenance:
    observed_directly_by: <agent_instance_id>
    reproduced_by: [<agent_instance_id>, ...]
    workspace: <path>
    commit: <sha | null>
  metadata:
    timestamp: <ISO-8601>
    causal_parent_id: <message_id | null>
    content_hash: <sha256 of normalized text>

```

Critical distinction: Fields in the **sender**, **addressee**, and **provenance** blocks are obtained mechanically from transport metadata and deterministic classification. Fields in the **speech_act** block may require semantic inference (LLM classifier or MeTTa rules), but they are clearly labeled as inferred, not observed.

The ledger is bounded: it retains only the most recent N messages per chat (default $N = 200$), with older entries archived to long-term memory. It is available as structured context to the model, replacing the current unstructured conversation history for routing decisions.

5 Implementation roadmap

5.1 Phase 1: Transport identity and routing (highest impact, lowest complexity)

Goal: Eliminate prose-based identity inference and misattribution.

Changes:

- Add **SelfContext** and **InboundEnvelope** as structured metadata injected into every model turn.
- Implement the deterministic addressee classifier (Layer 1) as a pre-model gateway function.
- Gate model generation on classification:
 - **DIRECT**: generate normally.
 - **SECONDARY**: generate with a system instruction to contribute only if there is a substantive correction; default to **SUPPRESS**.
 - **GROUP**: generate with a system instruction to respond only if the message is genuinely useful; default to **SUPPRESS**.
 - **INCIDENTAL**: skip generation entirely; return **SUPPRESS**.

Expected impact: Eliminates ~60% of misattribution-driven noise (Failures 1 and 2).

Effort: Low. Metadata is already available from Telegram; this is plumbing, not new ML.

Risk: Minimal. Incorrect classification is debuggable from the envelope, not from prose.

5.2 Phase 2: Runtime trace suppression and gateway silence

Goal: Stop internal runtime activity from reaching the chat channel.

Changes:

- Route tool-use planning strings, exec status, fallback notices, and heartbeat activity to an operator log, never to Telegram.
- Implement **SUPPRESS** as a gateway-intercepted control result that produces zero messages.

- Add a validator that rejects any proposed message containing `NO_REPLY`, “Staying quiet”, or a full-body explanation of why no reply is warranted.
- Add a validator that rejects any proposed message that is purely an acknowledgement (“Noted”, “Acknowledged”, “Got it”) with no additional substantive content.

Expected impact: Eliminates ~25% of channel noise (Failures 3 and 4) and removes the primary fuel for feedback loops.

Effort: Low-medium. Requires gateway changes to intercept and route, but no model changes.

Risk: Low. Suppressed messages are logged for debugging. If a valid message is accidentally suppressed, it can be recovered from the operator log.

5.3 Phase 3: Loop suppression and deduplication

Goal: Prevent repeated emission of identical content and mutual acknowledgement cascades.

Changes:

- Implement the `RoomLedger` with content hashing, causal parent IDs, and the four suppression rules from Layer 4.
- Add an acknowledgement cap: at most one acknowledgement message per 5 minutes per agent per chat.
- Add status-echo detection: messages matching the pattern “staying quiet / all work complete / noted / acknowledged” with no other substantive content are classified as status echoes and do not trigger sibling wake events.

Expected impact: Eliminates the feedback-loop pathology (Failure 5) and prevents the “all work complete / staying quiet” cascade observed in the transcript.

Effort: Medium. Requires persistent per-chat state in the gateway.

Risk: Low-medium. Dedup thresholds must be tuned. Too aggressive could suppress a legitimately repeated result; too lenient allows echoes. The content hash should normalize whitespace and case but preserve semantic content.

5.4 Phase 4: Session and task isolation

Goal: Prevent context leakage across chats and tasks.

Changes:

- Key task continuations by (`account`, `chat_id`, `thread_id`, `requesting_message_id`, `responsible_agent`).
- A completion event is publishable only into its originating conversation unless explicitly rerouted.
- Remove shared global “current task” state; replace with per-conversation task handles.
- Add an observation-scope tag to every agent report: `observed_by`, `workspace`, `commit`, `reproduced_by`.

Expected impact: Eliminates non-sequitur responses from wrong contexts (Failure 6) and epistemic scope confusion (Failure 7).

Effort: Medium. Requires changes to session routing and task continuation handling in both `OpenClaw` and `OmegaClaw`.

Risk: Medium. Must not break legitimate cross-chat task delegation. The explicit reroute mechanism is the safety valve.

5.5 Phase 5: Speech-act room ledger

Goal: Provide structured short-term conversation memory for better routing and contextual awareness.

Changes:

- Implement the **SpeechActRecord** schema from Layer 6.
- Populate mechanical fields (sender, addressee, provenance) from the envelope and deterministic classifier.
- Populate semantic fields (speech act type, content summary, expected respondent) via a lightweight LLM classifier or MeTTa rules.
- Expose the ledger as structured context to the model, replacing raw conversation history for routing decisions.
- Bound the ledger to $N = 200$ recent messages per chat.

Expected impact: Improves contextual awareness for ambiguous group-addressed messages and enables future features (e.g., “has this question already been answered?” detection).

Effort: Medium-high. Requires schema design, classifier implementation, and integration with both agent platforms.

Risk: Medium. Semantic classification can be wrong; the mechanical fields must remain authoritative. The ledger is an aid to routing, not a replacement for the deterministic classifier.

5.6 Phase 6: Social role configuration

Goal: Make activation policy explicit and configurable per agent.

Changes:

- Add **SocialRole** configuration to both OpenClaw and OmegaClaw agent manifests.
- Default ProtoCosmoBot and ProtoMegaBot to **always-attending**.
- Allow other bots to declare **special-occasions** with custom wake events.
- Even **always-attending** bots use the addressee classifier to determine whether to *speak*; “always attending” means “always reading,” not “always responding.”

Expected impact: Supports adding specialized bots without flooding the channel, and makes the activation policy auditable rather than emergent.

Effort: Low. Configuration change, not new infrastructure.

Risk: Minimal.

6 Phased rollout summary

Phase	Focus	Impact	Effort	Dependencies
1	Transport identity & addressee classifier	60% of misattribution	Low	None
2	Runtime trace suppression & gateway silence	25% of noise	Low-Med	None
3	Loop suppression & deduplication	Feedback loops	Medium	Phase 2
4	Session & task isolation	Cross-chat leakage	Medium	Phase 1
5	Speech-act room ledger	Contextual awareness	Med-High	Phases 1–4
6	Social role configuration	Scalable bot roles	Low	Phase 1

Table 1: Phased rollout. Phases 1 and 2 are independent and can proceed in parallel. Phase 3 depends on 2 (suppression must exist before deduplication adds value). Phase 5 depends on 1–4 being stable.

7 Open questions for external review

1. **Hard vs. soft addressee gate.** Should a `SECONDARY` or `INCIDENTAL` classification mechanically block generation, or should it produce a strong system instruction that the model may override? This design proposes hard gating with an explicit logged `SECONDARY_CONTRIBUTION` override, but the threshold for the override needs review.
2. **Acknowledgement cap window.** Five minutes is a placeholder. Too short and legitimate follow-ups are suppressed; too long and the feedback loop persists. Should it be adaptive (shorter for high-traffic chats, longer for low-traffic)?
3. **Status-echo detection heuristics.** The pattern list (“staying quiet”, “all work complete”, “noted”, “acknowledged”) is English-specific and brittle. Should this be a learned classifier or a configurable pattern set? What about non-English chats?
4. **Ledger size ($N = 200$).** Is 200 messages sufficient for routing decisions in a high-traffic philosophical discussion? Should the ledger retain semantic summaries of older messages rather than discarding them entirely?
5. **Cross-platform applicability.** The design is framed around Telegram metadata (mention entities, reply-to, bot user IDs). How much of it transfers to other chat platforms (Discord, Slack, Matrix) that have different metadata availability?
6. **Observation-scope provenance.** The `observed_by` / `reproduced_by` tags are proposed for agent reports. Should these be mandatory for all agent-to-agent assertions, or only for claims about filesystem/repos/code state?
7. **Speech-act classifier implementation.** Should the semantic fields of the `SpeechActRecord` be populated by a lightweight LLM call, by deterministic MeTTa rules, or by a hybrid? The LLM path is more accurate but adds latency and cost; the MeTTa path is faster but may require new rule development.

8 What this design does not address

- **Model capability differences.** Some of the identity confusion arose because different models (GLM-5.2, GPT-5.6-terra) have different instruction-following and context-tracking abilities. This design assumes the model can follow structured metadata; models that cannot may need additional prompting guardrails.
- **Cross-agent memory coherence.** The design isolates task context per conversation but does not address the deeper problem of how two agents with different filesystems and memory stores can make compatible claims about shared artifacts. The observation-scope tags are a provenance aid, not a coherence mechanism.
- **Human participant modeling.** The design treats all `is_bot=false` senders as “human” without distinguishing Ben (owner) from other human participants. Authorization and priority differences between human participants are out of scope here.
- **Multi-chat federation.** If the same agents participate in multiple Telegram chats, the design treats each chat independently. Whether a speech-act ledger should federate across chats (e.g., “this question was already answered in another room”) is deferred.

9 Success criteria

The design is successful if, after deployment:

1. No agent misattributes a message that contains an explicit Telegram `@mention` or reply-to target. (Failure 2 eliminated.)
2. No runtime trace, tool-use planning string, or control decision appears as a Telegram message. (Failure 3 eliminated.)
3. No message containing `NO_REPLY` or equivalent silence explanation reaches Telegram. (Failure 4 eliminated.)
4. The mutual acknowledgement cascade (“all work complete / staying quiet” loop) does not occur even when both agents are active and idle. (Failure 5 eliminated.)
5. An agent asked a question in a different channel does not produce a response from a pending task in another channel. (Failure 6 eliminated.)
6. An `always-attending` bot reads every message but speaks only when it has a direct response, a substantive correction, or a genuinely useful group contribution. (Social role enforced.)
7. The speech-act ledger correctly classifies $\geq 95\%$ of messages by addressee and $\geq 80\%$ by speech-act type, as measured against a labeled sample.